Subject: Information Security Lab (ICT 3141) **Date:** ____ - ____ - 2026

Semester, Branch, Section: 5th Sem CCE _____

Lab End-Sem Exam: 20 Marks **Duration: 90 min**

Name: _____ **Reg. No:** _____ **Roll No:** _____

SECURE GRANT EXPENSE PORTAL

Scenario:

A research institute manages grant expenses through an online portal. Researchers submit grant details, reviewers check their integrity and authenticity, and the finance officer computes the total grant expenditure. The system must keep grant data confidential and verifiable, and the total must be computed without decrypting individual amounts.

Requirements:

Researcher side: The researcher enters a Grant ID, Grant Description and Grant Amount. Before storing, the system:

- Encrypts the grant description with AES-192.
- Encrypts the grant amount with the Paillier cryptosystem.
- Computes an MD5 hash of the grant record.
- Signs the hash with an RSA digital signature.
- The researcher can submit grant details, and view all submitted records, including those entered earlier.

Reviewer side: The reviewer verifies the RSA signature and recomputes the MD5 hash to detect tampering. The reviewer can search the stored records by index (Grant ID) without exposing the encrypted contents. The reviewer can also view the AES-192 decrypted grant description as a hexadecimal dump.

Finance Officer side: The finance officer homomorphically aggregates the Paillier-encrypted amounts of the selected grants. The officer then verifies the signatures and decrypts only the final aggregate, never individual amounts.

Task:

Write a menu-driven Python program implementing the above roles and functions. Use role-based menus and cryptographic operations for confidentiality, integrity and authenticity. Store the encrypted, hashed and signed records persistently (file or database), so that earlier entries remain visible after a restart. Handle invalid input and tampered records gracefully. Display outputs for all functions.

Note: If the Python code is not menu-driven, the total marks obtained will be reduced to half.

Sample Input-Output:

Illustrative values only; long hex strings are abbreviated. Paillier n = 3127, g = 3128.

```
MAIN MENU: 1 Researcher | 2 Reviewer | 3 Finance Officer | 4 Exit
Input Choice: 1
RESEARCHER MENU: 1 Submit grant details | 2 View submitted details | 3 Back
Input Choice: 1; Grant ID: G001; Description: Lab equipment; Amount: 500
Output AES-192 CT(hex): [abbreviated]
Output Paillier amount CT(hex): [abbreviated]
Output MD5(hex): [abbreviated]
Output RSA signature: [abbreviated]
Output: G001 submitted and stored
Input Choice: 1; Grant ID: G002; Description: Field travel; Amount: 700
Output: G002 submitted and stored
Input Choice: 2
Output: G001 | CT: [..] | Paillier CT: [..] | MD5: [..]
        G002 | CT: [..] | Paillier CT: [..] | MD5: [..]
        (all earlier records also shown)
Input Choice: 3 -> MAIN MENU; Input Choice: 2

REVIEWER MENU: 1 Verify | 2 Search index | 3 View decrypted hex | 4 Back
Input Choice: 1; Grant ID: G001
Output: RSA signature valid; MD5 matches; G001 verified
Input Choice: 2; Grant ID: G002
Output: Record found: G002 (contents remain encrypted)
Input Choice: 3; Grant ID: G001
Output: AES-192 decrypted text: Lab equipment
Output: Decrypted hex: 4c616220657175697706d656e74
Input Choice: 3; Grant ID: G002
Output: Decrypted hex: 4669656c642074726176656c
Input Choice: 4 -> MAIN MENU; Input Choice: 3

FINANCE MENU: 1 Aggregate | 2 Verify | 3 Back
Input Choice: 1; Grant IDs: G001, G002
Output: Product of Paillier ciphertexts mod n^2 = [abbreviated]
Input Choice: 2
Output: Signatures valid; aggregate decrypted = 1200
Output: Individual amounts not decrypted
Input Choice: 3 -> MAIN MENU; Input Choice: 4 -> Exit
```

Evaluation Scheme:

Component	Criteria	Done
Writeup (5)	Python functionalities for grant entry, role menus and access control	
	Python functionalities for AES-192 encryption/decryption and MD5	
	Python functionalities for hexadecimal display of decrypted text	
	Python functionalities for Paillier encryption and aggregation	
	Python functionalities for RSA signing and verification	
	Python functionalities for search index and persistent storage	

Component	Criteria	Done
Execution (15)	**Researcher:** Enter Grant ID, description and amount	
	Researcher: Encrypt description using AES-192	
	Researcher: Encrypt amount using Paillier	
	Researcher: Compute MD5 hash of the record	
	Researcher: Create RSA signature	
	Researcher: View all submitted records, including earlier ones	
	Reviewer: Verify RSA signature	
	Reviewer: Recompute and compare MD5 hash	
	Reviewer: Search records by index	
	Reviewer: Decrypt description with AES-192 and display it in hexadecimal	
	Finance: Aggregate Paillier ciphertexts homomorphically	
	Finance: Verify signatures before aggregation	
	Finance: Decrypt only the aggregate total	
	All roles: Enforce role-based menus and protect stored records	
	All roles: Handle invalid input and tampered records	
	All roles: Display the specified outputs in a complete run	

Remarks: _____